



Paradigmas de la programación.

Universidad Autónoma del Estado del México.

Licenciatura en ingeniería en Computación.

Tercer Semestre

Otoño 2024.

Maestro. Julio Alberto de la teja López

Alumno: Alan Osornio Garcia

ICO 28.

13/01/2025.

Centro Universitario UAEM Atlacomulco.

```

PS C:\Users\elgat\Music\PYTHON> & C:/Users/elgat/Music/PYTHON/.venv/Scripts/python.exe c:/Users/elgat/Music/PYTHON/TITULO.PY
Cliente Juan Pérez registrado exitosamente.
Datos válidos. Creando pedido...
Pedido creado exitosamente.
Comprobante emitido: {'cliente': 'Juan Pérez', 'pedido': 'Tacos al pastor'}
Datos válidos. Creando pedido...
Pedido creado exitosamente.
Comprobante emitido: {'cliente': 'Juan Pérez', 'pedido': 'Tacos al pastor'}
Comprobante emitido: {'cliente': 'Juan Pérez', 'pedido': 'Tacos al pastor'}
Preparando el plato: Tacos al pastor
Cliente Juan Pérez registrado exitosamente.
Datos válidos. Creando pedido...
Pedido creado exitosamente.
Comprobante emitido: {'cliente': 'Juan Pérez', 'pedido': 'Enchiladas verdes'}
Preparando el plato: Enchiladas verdes
PS C:\Users\elgat\Music\PYTHON> ^C
PS C:\Users\elgat\Music\PYTHON> & C:/Users/elgat/Music/PYTHON/.venv/Scripts/python.exe c:/Users/elgat/Music/PYTHON/TITULO.PY
Cliente Juan Pérez registrado exitosamente.
Datos válidos. Creando pedido...
Pedido creado exitosamente.
Comprobante emitido: {'cliente': 'Juan Pérez', 'pedido': 'Tacos al pastor'}
Preparando el plato: Tacos al pastor
Cliente Juan Pérez registrado exitosamente.
Error: El pedido no puede estar vacío.
PS C:\Users\elgat\Music\PYTHON>

```

⊗ Unable to s
Please open

<https://github.com/AlanOsornio/desktop-tutorial.git>

Clase base: Persona

class Persona:

```

    def __init__(self, nombre):

        self.__nombre = nombre # Atributo encapsulado

```

```

    def get_nombre(self):

        return self.__nombre

```

Clase Cliente (Hereda de Persona)

class Cliente(Persona):

```

    def realizar_pedido(self, pedido):

        print(f"{self.get_nombre()} realiza un pedido: {pedido}")

        return pedido

```

Clase Sistema para validar y gestionar pedidos

class Sistema:

def __init__(self):

self.pedidos = [] # Lista para almacenar pedidos

self.clientes = [] # Lista para almacenar clientes registrados

def registrar_cliente(self, cliente: Cliente):

self.clientes.append(cliente)

print(f"Cliente {cliente.get_nombre()} registrado exitosamente.")

def validar_datos(self, pedido):

if not pedido:

raise ValueError("Error: El pedido no puede estar vacío.")

print("Datos válidos. Creando pedido...")

return True

def crear_pedido(self, cliente, pedido):

if cliente not in self.clientes:

raise ValueError("Error: Cliente no registrado en el sistema.")

self.pedidos.append({"cliente": cliente.get_nombre(), "pedido": pedido})

print("Pedido creado exitosamente.")

return {"cliente": cliente.get_nombre(), "pedido": pedido}

Clase Cocina para procesar los pedidos

class Cocina:

def preparar_plato(self, pedido):

print(f"Preparando el plato: {pedido}")

```
# Clase Principal para la ejecución del flujo
```

```
class Restaurante:
```

```
    def __init__(self):
```

```
        self.sistema = Sistema()
```

```
        self.cocina = Cocina()
```

```
    def procesar_pedido(self, cliente, pedido):
```

```
        try:
```

```
            self.sistema.registrar_cliente(cliente) # Registrar cliente al sistema
```

```
            if self.sistema.validar_datos(pedido):
```

```
                pedido_creado = self.sistema.crear_pedido(cliente, pedido)
```

```
                print(f"Comprobante emitido: {pedido_creado}")
```

```
                self.cocina.preparar_plato(pedido)
```

```
        except ValueError as e:
```

```
            print(e)
```

```
# Ejecución
```

```
if __name__ == "__main__":
```

```
    cliente = Cliente("Juan Pérez")
```

```
    restaurante = Restaurante()
```

```
# Flujo de creación de pedido
```

```
    restaurante.procesar_pedido(cliente, "Tacos al pastor")
```

```
    restaurante.procesar_pedido(cliente, "Enchiladas verdes")
```